

Count Digits ASN

①

```
def ASN(n, pow, asn, temp)
    if 0 <= 0 -> T
    return asn = temp
    153 == 153
```

n = 0
pow = 3
asn = 153
temp = 153

```
def ASN(n, pow, asn, temp)
    if 1 <= 0 - F
    base = 1 % 10
    asn = 153 + (1 * x 3)
    n = 1 // 10
```

n = 1 0
pow = 3
asn = 152 152
temp = 153

```
return is ASN(0, 3, 153, 153)
```

```
def ASN(n, pow, asn, temp)
    if 15 <= 0 - F
    base = 15 % 10
    asn = 27 + (5 * x 3)
    n = 15 // 10
```

n = 15 1
pow = 3
asn = 27 152
temp = 153

```
return is ASN(1, 3, 152, 153)
```

```
def ASN(n, pow, asn, temp)
    if 153 <= 0 - F
    base = 153 % 10
    asn = 0 + (3 * x 3)
    n = 153 // 10
```

n = 153 15
pow = 3
asn = 0 27
temp = 153

```
return is ASN(15, 3, 27, 153)
```

STACK

```
def ASN(153, 3, 0, 153)
```

Sum of natural numbers

$n=3$

```
def SN(n, Sum)
    if 0 <= 0 → F
    return Sum
    6
```

$n=0$
 $Sum=6$

```
def SN(n, Sum)
    if 1 <= 0 → F
    Sum = 5 + 1
```

$n=1$
 $Sum=5$

```
    n = 1 - 1
    return SN(0, 6)
```

```
def SN(n, Sum)
    if 2 <= 0 → F
    Sum = 3 + 2
```

$n=2$
 $Sum=3$

```
    n = 2 - 1
    return SN(1, 5)
```

```
def SN(n, Sum)
    if 3 <= 0 → F
    Sum = 0 + 3
    n = 3 - 1
```

$n=3$
 $Sum=0$

```
    return SN(2, 3)
```

STACK

```
return SN(3, 0)
```

$n=3$
 $3+2+1=6$

Sum of digit

$n=481$

Sum of digit
 $n = 5431$

def $dn(n, sum)$
 if $0 \leq 0 \rightarrow [T]$
 return sum
 13

def $dn(n, sum)$
 if $5 \leq 0$
 $d = 5 \% 10$
 $sum = 8 + 5$
 $n = 5431 / 10$
 return $dn(0, 13)$

$n = 50$
 $sum = 8 + 13$
 $d = 45$

def $dn(n, sum)$
 if $54 \leq 0 \rightarrow F$
 $d = 54 \% 10$
 $sum = 4 + 4$
 $n = 5431 / 10$
 return $dn(5, 8)$

$n = 545$
 $sum = 48$
 $d = 34$

def $dn(n, sum)$
 if $543 \leq 0 \rightarrow F$
 $d = 543 \% 10$
 $sum = 1 + 3$
 $n = 5431 / 10$
 return $dn(54, 4)$

$n = 54354$
 $sum = 44$
 $d = 43$

def $dn(n, sum)$
 if $5431 \leq 0 \rightarrow F$
 $d = 5431 \% 10$
 $sum = 0 + 1$
 $n = 5431 / 10$
 return $dn(543, 1)$

$n = 5431543$
 $sum = 01$
 $d = 01$

def $dn(5431, 0)$
 STACK

$$5 + 4 + 3 + 1 = 13$$

count fact

```
def cn(n, i, countf)
  if 5 > 4 -> T
  return countf
```

n = 4
countf = 3

```
def cn(n, i, countf)
  if 4 > 4 -> F
  if 4 % 2 == 0 F
  countf + 1
```

n = 4
i = ~~4~~ 5
countf = ~~2~~ 3

```
return cn(4, 4, 3)
```

```
def cn(n, i, countf)
  if 3 > 4 -> F
  if 3 % 2 == 0 F
```

n = 4
i = ~~4~~ 3
countf = 2

```
return cn(4, 3, 2)
```

```
def cn(n, i, countf)
  if 2 > 4 -> F
  if 2 % 2 == 0
```

n = 4
i = ~~3~~ 2
countf = 1 2

```
return cn(4, 2, 2)
```

```
def cn(n, i, countf)
  if 1 > 4 -> F
  if 1 % 1 == 0
```

n = 4
i = ~~2~~ 1
countf = 0 1

```
return cn(4, 1, 1)
```

```
return cn(4, 0, 0)
```

count fact

def cn(n, i)

n = 3

Primp
(2)

```
def PC n, i, count
  if (6 > 5) - T
  return 9 == 2
```

```
def PC n, i, count
  if (5 > 5) - F
  if 5 % 5 == 0 - T
    count += 1
  return PC(5, 6, 2) ←
```

$n = 6$
 $i = 4$
 $count = 9$

```
def PC n, i, count
  if (4 > 5) - F
  if 5 % 4 == 0 -> F
  return PC(5, 5, 1) ←
```

$n = 6$
 $i = 4$
 $count = 1$

```
def PC n, i, count
  if 3 > 5 - F
  if 5 % 2 == 0 -> F
  return PC(5, 4, 1) ←
```

$n = 6$
 $i = 3$
 $count = 1$

```
def PC n, i, count
  if 2 > 5 - F
  if 5 % 2 == 0 - F
  count += 1
  return PC(5, 3, 1) ←
```

$n = 6$
 $i = 2$
 $count = 1$

```
def PC n, i, count
  if 1 > 5 - F
  if 5 % 1 == 0 -> T
    count += 1
  return PC(5, 2, 1) ←
```

$n = 5$
 $i = 1 + 1$
 $count = 0$

```
PC(5, 1, 0) ←
```

Constructors Square Root Logic

(3)

```
def f(m, i)
    if 5 * 5 > m - F
    return
```

m = 12
i = 4

```
def f(m, i)
    if 3 * 3 > m - F
    if 12 % 3 == 0 - T
        print(3)
    if (3! = 12 // 3) - T
        print(12 // 3)
```

m = 12
i = 4
OP = 3, 4

return f(12, 4)

```
def f(m, i)
    if 2 * 2 > m - F
    if 12 % 2 == 0 - T
        print(2)
    if (2! = 12 // 2) - T
        print(12 // 2)
```

m = 12
~~Recursion~~
i = 3
OP = 2, 6

return f(12, 3)

```
def f(m, i)
    if 1 * 1 > m - F
    if 12 % 1 == 0 - T
        print(1)
    if (1! = 12 // 1) - T
        print(12 // 1)
```

m = 12
~~Recursion~~
i = 2
O/P = 1, 12

return f(12, 2)

return f(12, 1)
Stack

m = 12 = 1, 12, 2, 6, 3, 4

④ Square root log.c count facts

```
def cf(n, i, count)
  if 5 * 5 > 16 - F
  return count = 5
```

```
def cf(n, i, count)      n = 16
                          i = 4
  if 4 * 4 > 16 - F
  if 16 % 4 == 0 - T
    count + 1
  return (16, 4, 5)
```

```
def cf(n, i, count)      n = 16
                          i = 3
  if 3 * 3 > 16 - F
  if 16 % 3 == 0 - F
  return cf(16, 3, 4)
```

```
def cf(n, i, count)      n = 16
                          i = 2
  if 2 * 2 > 16 - F
  if 16 % 2 == 0 - T
    count + 1
  if 2 != 16 // 2
    count + 2
  return cf(16, 3, 4)
```

```
def cf(n, i, count)      n = 16
                          i = 1
  if 1 * 1 > 16 - F
  if 16 % 1 == 0 - T
    count + 1
  if 1 != 16 // 1 - T
    count + 2
  return cf(16, 2, 2)
```

cf(16, 1, 0)

Stack

n = 16 count = 5

5) Square root logic Prime numbers

```
def p(n, i, count)
    if 4*4 > 13 - T
    return 13, 3, 0
    return 2 == 2
```

```
def p(n, i, count)
    if 3*3 > 13 - F
    if 13 % 3 == 0 - F
    return p(13, 3, 2)
```

n = 13
i = 3
count = 2

```
def p(n, i, count)
    if 2*2 > n: - F
    if 13 % 2 == 0 - F
    return(13, 2, 2)
```

n = 13
i = 2
count = 2

```
def p(n, i, count)
    if 1*1 > n: - F
    if 13 % 1 == 0: - T
    count += 1
    if 1*1 = 13 // 1 - T
    count += 1
    return(13, 1, 2)
```

n = 13
i = 1
count = 2 + 2

STACK

P(13, 1, 0)

2 == 2

13 is prime

(13, 1, 0)

⑥ GCD

```
def gcd(m1, m2, i, hcf, lower)
    if 5 > 4 - F
    return hcf
    2
```

```
def gcd(m1, m2, i, hcf, lower)
    if 4 > 4 -
    if 10 % 4 == 0 and 10 % 4 == 0
        → F
    hcf = 2
    lower = 4
```

return gcd(10, 4, 5, 2, 4)

```
def gcd(m1, m2, i, hcf, lower)
    if 3 > 4 - F
    if 10 % 3 == 0 and 10 % 3 == 0:
        → F
    hcf = 2
    lower = 4
```

return gcd(10, 4, 4, 2, 4)

```
def gcd(m1, m2, i, hcf, lower)
    if 2 > 4 - F
    if 10 % 2 == 0 and 4 % 2 == 0:
        hcf = 2
    hcf = 2
    lower = 4
```

return gcd(10, 4, 3, 2, 4)

```
def gcd(m1, m2, i, hcf, lower):
    if 1 > 4 - F
    if 10 % 1 == 0 and 4 % 1 == 0:
        hcf = 1
    hcf = 1
    lower = 4
```

return gcd(10, 4, 2, 1, 4)

gcd(10, 4, 1, 0, 4)

m1 = 10
m2 = 4
GCD = 2

7

isPrime

$n1 = 3$
 $n2 = 15$

$gcd = 2$

not prime

```
def cp(m1, n2, i, hcf, lower)
    if  $i > 3 \rightarrow T$ 
    return  $hcf = 1$ 
            $g = 1$ 
```

```
def cp(m1, n2, i, hcf, lower)
    if  $3 > 3 \rightarrow F$ 
    if  $3 \% 3 == 0$  and  $15 \% 3 == 0$ 
         $hcf = 3$ 
```

$n1 = 3$
 $n2 = 15$
 $i = 3$
 $hcf = 3$
 $lower = 3$

```
return cp(3, 15, 1, 3)
```

```
def cp(m1, n2, i, hcf, lower)
    if  $3 > 3 \rightarrow F$ 
    if  $3 \% 3 == 0$  and  $15 \% 3 == 0$ 
         $hcf = 3$ 
```

$n1 = 3$
 $n2 = 15$
 $i = 3$
 $hcf = 3$
 $lower = 3$

```
return cp(3, 15, 3, 1, 3)
```

```
def cp(m1, n2, i, hcf, lower)
    if  $1 > 3 \rightarrow F$ 
    if  $3 \% 1 == 0$  and  $15 \% 1 == 0$ 
         $hcf = 1$ 
```

$n1 = 3$
 $n2 = 15$
 $i = 1$
 $hcf = 1$
 $lower = 3$

```
return cp(3, 15, 2, 1, 3)
```

```
cp(3, 15, 1, 0, 3)
```

$n1 = 3$
 $n2 = 15$

$(3, 15)$ is not coprime

8 Euclidean algorithm

GCD

$$n_1 = 20$$

$$n_2 = 6$$

$$\text{gcd} = 2$$

def gcd(n1, n2)

if n1 == 0 → F

return n2

def gcd(n1, n2)

if 20 == 0 → F

if 20 < 6 → T

n1, n2, 6, 2

return gcd(6 % 2, 2)

def gcd(n1, n2)

if 20 == 0 → F

if 20 < 6 → F

return gcd(20 % 6, 6)

gcd(20, 6)

new
(2, 6)
(0, 2)

n1 = 20

n2 = 6

hcf = (2, 6)

9 Euclidean algorithm

Coprime

$$n1 = 18$$

$$n2 = 26$$

$$\text{gcd} = 2$$

not coprime

def p(n1, n2)

if $n2 == 0$ T

return $2 == 1$

False

not coprime

def p(n1, n2)

if $n2 == 0$

if $2 < 18$

$n1, n2 = 8, 2$

return(p($8 \% 2, 2$))

(2, 8)
(0, 2)

def p(n1, n2)

if $8 == 0$ - F

if $8 < 18$ - T

$n1, n2 = 18, 8$

return(p($18 \% 8, 8$))

8, 18
(2, 8)

def p(n1, n2)

if $18 == 0$ - F

if $18 < 36$

$n1, n2 = 36, 18$

return(p($36 \% 18, 18$))

18, 36
(8, 18)

cop(18, 26)

STACK